

Utility Usage Prediction Tool

CodeVedX Internship Project

Submitted By

Name: Aarti Prajapati

Enrollment Number: 21231258

Course: B.Tech CSE (AI & ML), 7th Semester

College: IIMT University, Meerut ,Uttar Pradesh

Email : aartiainl3009@gmail.com

Project Theme

Design and Development of a Machine Learning-Based Utility Usage Prediction Tool using Data Analysis and Predictive Modeling

GitHub Repository

https://github.com/aartiprajapati-ai/CODEVEDX/tree/main/Utility_Usage_Prediction_Tool

Academic Year: 2026–2027

INTRODUCTION

The increasing demand for electricity and other utility resources has made efficient resource management an important aspect of modern life. Predicting utility usage in advance helps individuals and organizations monitor consumption, reduce wastage, and make informed decisions regarding energy management. Traditional methods of estimating utility usage are often manual and may not provide accurate results.

The **Utility Usage Prediction Tool** is a Machine Learning-based application developed to predict utility consumption using historical usage data. The system analyzes user-provided information and generates predictions through a trained Machine Learning model. It provides a simple console-based interface that enables users to add, update, and predict utility usage efficiently.

The project demonstrates the complete Machine Learning workflow, including data collection, preprocessing, model training, prediction, and result generation. It is developed using **Python** and various Machine Learning libraries to provide a practical solution for utility usage analysis.

The primary objective of this project is to demonstrate the practical implementation of Artificial Intelligence and Machine Learning for predicting utility usage and supporting better resource management.

PROBLEM STATEMENT

Managing utility consumption manually is often time-consuming and may result in inaccurate estimations. Without predictive analysis, it becomes difficult to estimate future utility usage and identify consumption patterns. This can lead to inefficient resource utilization and increased operational costs.

There is a need for an intelligent system that can analyze historical utility data and predict future usage accurately. The **Utility Usage Prediction Tool** addresses this problem by applying Machine Learning techniques to analyze utility data and generate reliable predictions. The system helps users make informed decisions and promotes efficient utilization of resources.

PROJECT OBJECTIVE

- The main objectives of the **Utility Usage Prediction Tool** are:
- To develop a Machine Learning-based utility usage prediction system.
- To analyze historical utility consumption data.
- To implement data preprocessing techniques for better prediction.
- To train a Machine Learning model for predicting utility usage.
- To provide a simple console-based interface for user interaction.
- To improve prediction accuracy using historical data.

- To demonstrate the practical implementation of Artificial Intelligence and Machine Learning.

EXPECTED OUTCOMES

- Accurate prediction of utility usage.
- Easy analysis of consumption data.
- Better resource management.
- Improved understanding of Machine Learning concepts.
- A user-friendly prediction system.

SCOPE OF THE PROJECT

The **Utility Usage Prediction Tool** is designed to predict utility consumption using historical usage data. The project provides a simple console-based application where users can manage utility records and obtain usage predictions through a Machine Learning model.

The current version focuses on CSV-based data storage and prediction using predefined input features. In the future, the system can be integrated with smart meters, cloud databases, IoT devices, and advanced Machine Learning algorithms for real-time utility monitoring and forecasting.

LIMITATIONS

- Prediction accuracy depends on the quality of historical data.
- The system uses a limited number of input features.
- Real-time utility data is not supported.
- The application is console-based and does not include a graphical user interface.
- External factors affecting utility consumption are not considered.
- The system is intended for educational and demonstration purposes.

SOFTWARE REQUIREMENTS

The following software tools and technologies were used to design, develop, and implement the **Utility Usage Prediction Tool**.

Software	Purpose
Python 3.x	Programming Language
Visual Studio Code	Code Editor and Development Environment
Jupyter Notebook / Google Colab	Model Development and Testing
Pandas	Data Loading and Preprocessing
NumPy	Numerical Computations
Scikit-learn	Machine Learning Model Development
Joblib	Model Saving and Loading
CSV Module	Reading and Writing Utility Data
Git & GitHub	Version Control and Project Hosting
Windows Command Prompt / PowerShell	Running the Console Application

HARDWARE REQUIREMENTS

The following hardware components are required to develop and run the **Utility Usage Prediction Tool** efficiently.

Hardware Component	Specification
Processor	Intel Core i3 or above (Intel Core i5 Recommended)
RAM	Minimum 4 GB (8 GB Recommended)
Storage	Minimum 500 MB Free Disk Space
Operating System	Windows 10 / Windows 11
Display	1366 × 768 Resolution or Higher
Internet Connection	Required for GitHub access, library installation, and project updates

PYTHON LIBRARIES USED

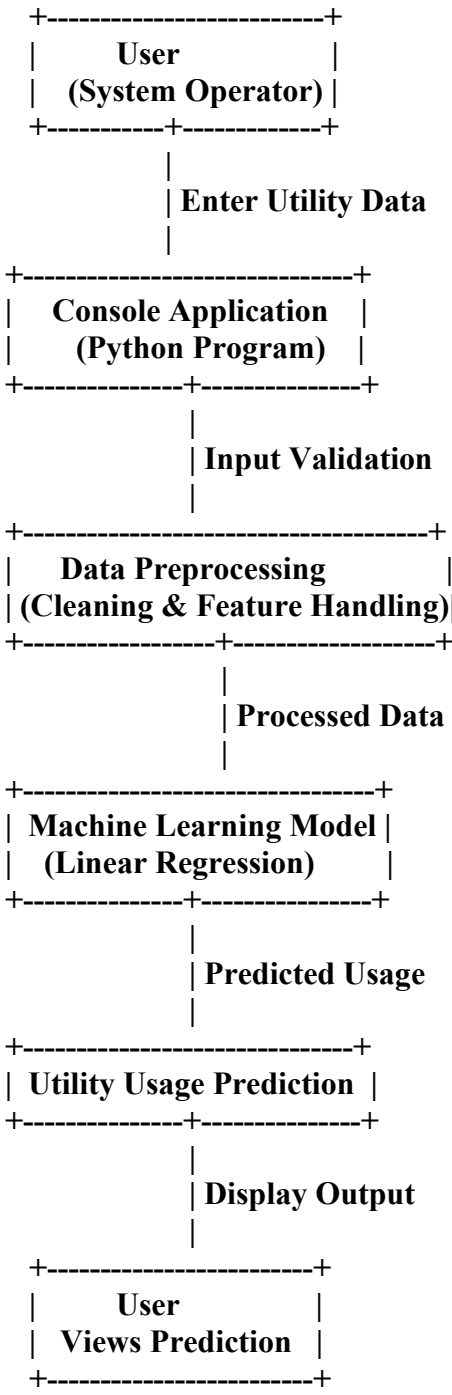
The **Utility Usage Prediction Tool** uses several Python libraries to perform data processing, machine learning, model persistence, and file handling. These libraries simplify the implementation of the prediction system and improve the overall efficiency of the application.

Library	Purpose
Pandas	Used for loading, reading, and preprocessing utility usage data from CSV files.
NumPy	Used for numerical computations and array operations.
Scikit-learn	Used for training the Machine Learning prediction model.
Joblib	Used for saving and loading the trained Machine Learning model.
CSV	Used for reading, writing, adding, and updating utility usage records.

SYSTEM ARCHITECTURE

The **Utility Usage Prediction Tool** follows a simple Machine Learning architecture for predicting utility consumption based on user-provided data. The user enters utility-related information through the console application. The input data is preprocessed and passed to the trained Machine Learning model. The model analyzes the input values and predicts the expected utility usage. Finally, the predicted result is displayed to the user. This architecture ensures efficient data processing, accurate predictions, and easy interaction with the application.

System Architecture Flow



WORKING OF THE SYSTEM

The **Utility Usage Prediction Tool** performs the following steps:

1. The user enters the required utility-related input values through the console application.
2. The application validates the entered data.
3. The input data is preprocessed and formatted for prediction.
4. The processed data is passed to the trained **Linear Regression** model.
5. The model analyzes the input values and predicts the expected utility usage.
6. The predicted utility usage is displayed on the screen.
7. The prediction can be used for better planning and efficient resource management.

DATASET DESCRIPTION

The **Utility Usage Prediction Tool** uses a structured dataset containing historical utility consumption records. The dataset is stored in **CSV (Comma-Separated Values)** format and is used to train the Machine Learning model for predicting future utility usage.

The dataset contains important attributes related to utility consumption, such as the number of units consumed, previous usage records, and billing information. These features help the Machine Learning model identify consumption patterns and generate accurate predictions.

Before training the model, the dataset is preprocessed by removing inconsistencies, selecting relevant features, and preparing the data for model training. A high-quality dataset improves the prediction accuracy and overall performance of the system.

Dataset Information

Attribute	Description
Dataset Name	Utility Usage Dataset
File Name	utility_usage.csv
File Format	CSV (Comma-Separated Values)
Dataset Type	Structured Tabular Dataset
Domain	Energy & Utility Management
Purpose	Predict Future Utility Usage

Dataset Schema

Column Name	Data Type	Description
Month	String	Month of utility usage
Previous_Usage	Float	Utility units consumed in the previous period
Current_Usage	Float	Current utility units consumed
Bill_Amount	Float	Total bill generated for the utility usage
Predicted_Usage	Float	Predicted utility usage generated by the model

Sample Dataset

Month	Previous Usage	Current Usage	Bill Amount	Predicted Usage
January	210	220	1650	218
February	220	230	1725	228
March	230	245	1835	243
April	245	250	1875	248
May	250	265	1980	262

DATA PREPROCESSING

Before training the Machine Learning model, the following preprocessing steps were performed:

- Loaded the dataset using the **Pandas** library.
- Checked the dataset for missing or inconsistent values.
- Selected relevant features required for prediction.
- Separated the input features (**X**) and target variable (**Y**).
- Split the dataset into training and testing datasets.
- Trained the Machine Learning model using the processed dataset.
- Saved the trained model using the **Joblib** library for future predictions.

METHODOLOGY

The **Utility Usage Prediction Tool** was developed using a systematic Machine Learning approach. The methodology includes data collection, preprocessing, model training, evaluation, and prediction. Each stage was carefully implemented to ensure accurate utility usage prediction.

Step 1: Data Collection

The utility usage dataset was collected and stored in **CSV format**. The dataset contains historical utility consumption records that are used to train the Machine Learning model.

Step 2: Data Loading

The dataset was loaded into Python using the **Pandas** library for further analysis and preprocessing.

```
import pandas as pd
```

```
df = pd.read_csv("utility_usage.csv")
```

Step 3: Data Preprocessing

Before training the Machine Learning model, the dataset was cleaned and prepared.

The preprocessing process included:

- Loading the dataset

- Checking for missing values
- Removing inconsistent data (if required)
- Selecting relevant input features
- Preparing the data for model training

Step 4: Feature Selection

The important input features were selected from the dataset to improve the prediction performance.

Typical features include:

- Previous Utility Usage
- Current Usage
- Billing Information
- Consumption History

These features were used to predict future utility usage.

Step 5: Train-Test Split

The dataset was divided into two parts:

- **Training Dataset – 80%**
- **Testing Dataset – 20%**

This allows the model to be evaluated on unseen data.

Step 6: Model Training

A **Linear Regression** model was used to train the prediction system.

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

The trained model learns the relationship between historical utility data and future utility usage.

Step 7: Model Evaluation

The trained model was evaluated using the testing dataset.

The predicted values were compared with the actual values to measure the model's prediction performance and reliability.

Step 8: Model Saving

The trained Machine Learning model was saved using the **Joblib** library.


```
import joblib
```

```
joblib.dump(model, "utility_usage_model.pkl")
```

Saving the model allows predictions to be made without retraining the model each time.

Step 9: Prediction

The saved model is loaded into the application. The user provides the required utility-related information, and the model predicts the expected utility usage based on the input values.

PROJECT IMPLEMENTATION

The **Utility Usage Prediction Tool** was implemented using **Python** and **Machine Learning** techniques. The project is designed in a modular manner, where each file performs a specific function such as data handling, model loading, prediction, and user interaction. The application uses a trained **Linear Regression** model to predict utility usage based on the input provided by the user.

The implementation consists of the following modules:

1. Dataset Module (utility_usage.csv)

The dataset module stores historical utility usage records used for training and testing the Machine Learning model.

Responsibilities:

- Store historical utility usage data.
- Provide input data for model training.
- Maintain data in CSV format for easy access.

2. Model Module (utility_usage_model.pkl)

The trained Machine Learning model is saved in a .pkl file using the **Joblib** library.

Responsibilities:

- Store the trained Linear Regression model.
- Load the trained model during application execution.
- Predict utility usage based on user input.

3. Application Module (app.py)

The app.py file acts as the main application.

Responsibilities:

- Load the trained Machine Learning model.
- Accept user input through the console application.
- Process the input data.
- Predict utility usage.
- Display the prediction result.

4. User Interaction Module

The system provides a simple console-based interface where users can enter utility-related information.

The application allows users to:

- Enter utility consumption details.
- View predicted utility usage.
- Analyze utility consumption patterns.

5. Prediction Process

The prediction process follows these steps:

1. The user enters utility-related input values.
2. The application validates the entered data.
3. The trained Machine Learning model processes the input.
4. The Linear Regression model predicts the expected utility usage.
5. The prediction result is displayed to the user.

RESULTS AND OUTPUT

The **Utility Usage Prediction Tool** was successfully developed and tested using a Machine Learning model. The application analyzes historical utility usage data and predicts future utility consumption based on user input.

The trained **Linear Regression** model generated reliable predictions, demonstrating the effectiveness of Machine Learning in utility usage forecasting. The console-based application provides a simple interface where users can enter utility-related values and instantly obtain prediction results.

The successful implementation of this project highlights the practical use of **Artificial Intelligence** and **Machine Learning** in resource management, enabling better planning and efficient utilization of utilities.

Output Screenshots

1. User Input

Description:

The user enters the required utility-related information, which is used by the Machine Learning model for prediction.

Screenshot:

The screenshot displays a code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project named 'UTILITY_USAGE_PREDICTION_TOOL' with files like 'utility_model.pkl', 'dataset_preview.png', 'folder_structure.png', 'model_training.png', 'prediction_output.png', 'app.py', 'README.md', 'requirements.txt', 'train_model.py', and 'UTILITY_USAGE_PREDICTION_TOOL'. The 'app.py' file is selected, showing the following Python code:

```

19
20
21     try:
22         voltage = float(input("Enter Voltage: "))
23         current = float(input("Enter Global Intensity: "))
24
25         prediction = model.predict([[voltage, current]])
26
27         print("\nPredicted Global Active Power:",
28               | round(prediction[0], 3), "kW")
29
30     except ValueError:
31         print("\nInvalid input! Please enter numeric values.")
32
33 elif choice == "2":
34
35     print("\nThank you for using Utility Usage Prediction Tool!")
36     break
37
38 if __name__ == '__main__':
39     main()
40

```

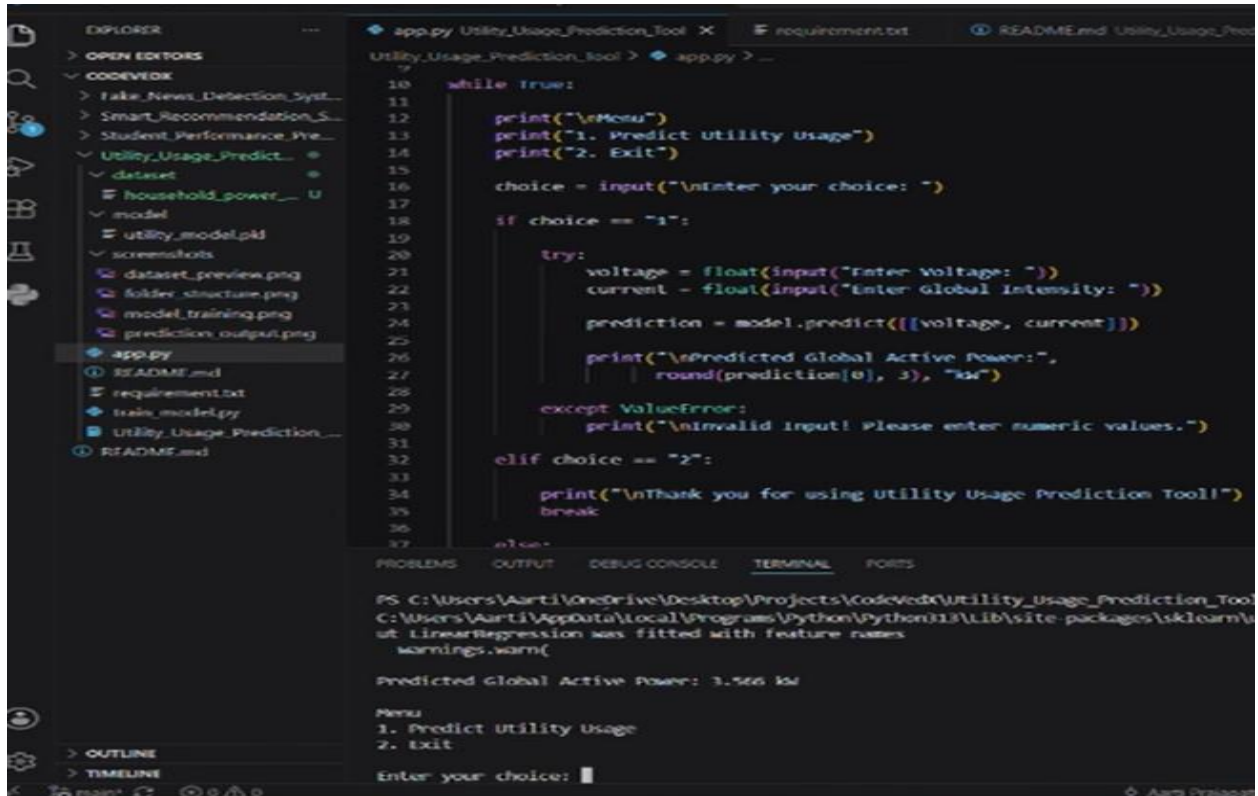
The terminal window shows the command prompt running the script: `PS C:\Users\Aarti\OneDrive\Desktop\Projects\CodeVedX\Utility_usage_Prediction_Tool> python app.py`. The output displays the 'Utility Usage Prediction Tool' menu, which includes '1. Predict Utility Usage' and '2. Exit'. The user has chosen option 1, and the program prompts for 'Enter Voltage: 240' and 'Enter Global Intensity: 10'.

2. Utility Usage Prediction Result

Description:

The trained Linear Regression model processes the input values and displays the predicted utility usage.

Screenshot:



The screenshot shows a VS Code editor with the file explorer on the left displaying a project structure. The main editor window shows the code for `app.py` in the `Utility_Usage_Prediction_Tool` directory. The code is a Python script that runs a while loop to provide a menu for predicting utility usage. The terminal at the bottom shows the execution output, including a warning about feature names and the predicted global active power.

```
10 while True:
11
12     print("\nMenu")
13     print("1. Predict utility Usage")
14     print("2. Exit")
15
16     choice = input("\nEnter your choice: ")
17
18     if choice == "1":
19
20         try:
21             voltage = float(input("Enter Voltage: "))
22             current = float(input("Enter Global Intensity: "))
23
24             prediction = model.predict([[voltage, current]])
25
26             print("\nPredicted Global Active Power:",
27                   round(prediction[0], 3), "kW")
28
29         except ValueError:
30             print("\nInvalid input! Please enter numeric values.")
31
32     elif choice == "2":
33
34         print("\nThank you for using Utility Usage Prediction Tool!")
35         break
36
37     else:
```

PS C:\Users\Aarti\OneDrive\Desktop\Projects\CodeVedx\Utility_Usage_Prediction_Tool> python app.py

C:\Users\Aarti\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\utils\linear_regression.py:111: UserWarning: LinearRegression was fitted with feature names

warnings.warn(

Predicted Global Active Power: 3.566 kW

Menu

1. Predict utility Usage

2. Exit

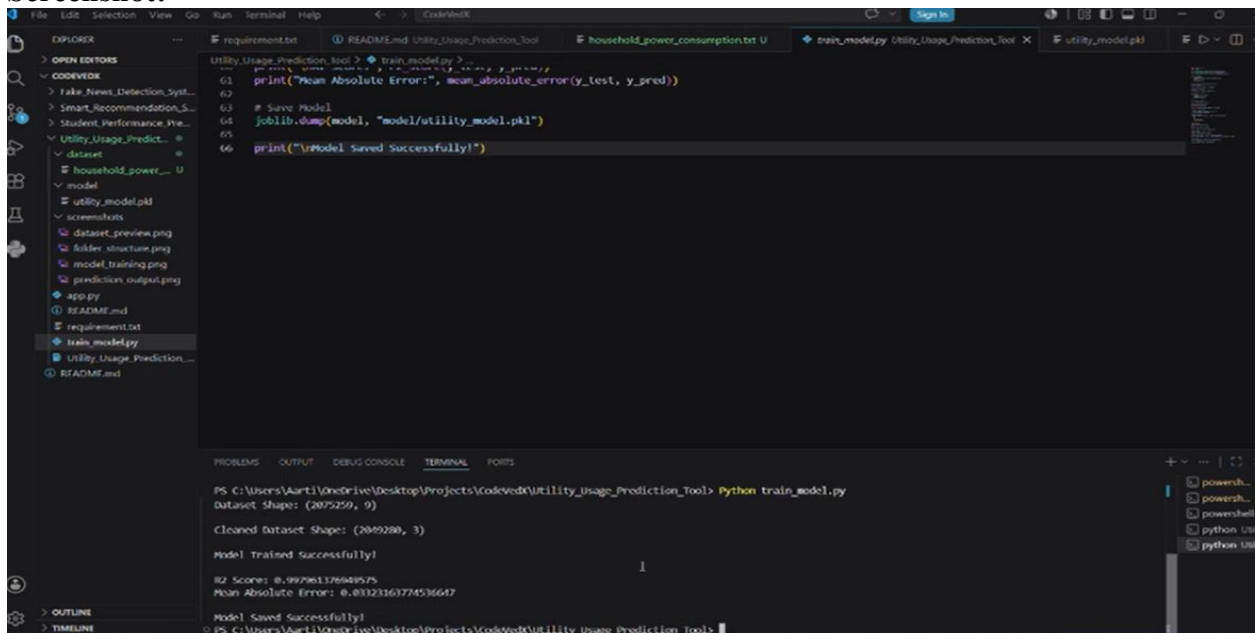
Enter your choice: 1

3. Model Loading

Description:

The saved Machine Learning model (`utility_usage_model.pkl`) is successfully loaded using the Joblib library before generating predictions.

Screenshot:



The screenshot shows a VS Code editor with the file explorer on the left displaying a project structure. The main editor window shows the code for `train_model.py` in the `Utility_Usage_Prediction_Tool` directory. The code is a Python script that loads a pre-trained model using Joblib and prints the mean absolute error.

```
1 print("Mean Absolute Error:", mean_absolute_error(y_test, y_pred))
2
3 # Save Model
4 joblib.dump(model, "model/utility_model.pkl")
5
6 print("\nModel Saved Successfully!")
```

PS C:\Users\Aarti\OneDrive\Desktop\Projects\CodeVedx\Utility_Usage_Prediction_Tool> python train_model.py

Dataset Shape: (2075259, 9)

Cleaned Dataset Shape: (2049280, 3)

Model Trained Successfully!

R2 Score: 0.997961376469576

Mean Absolute Error: 0.8323163774536647

Model Saved Successfully!

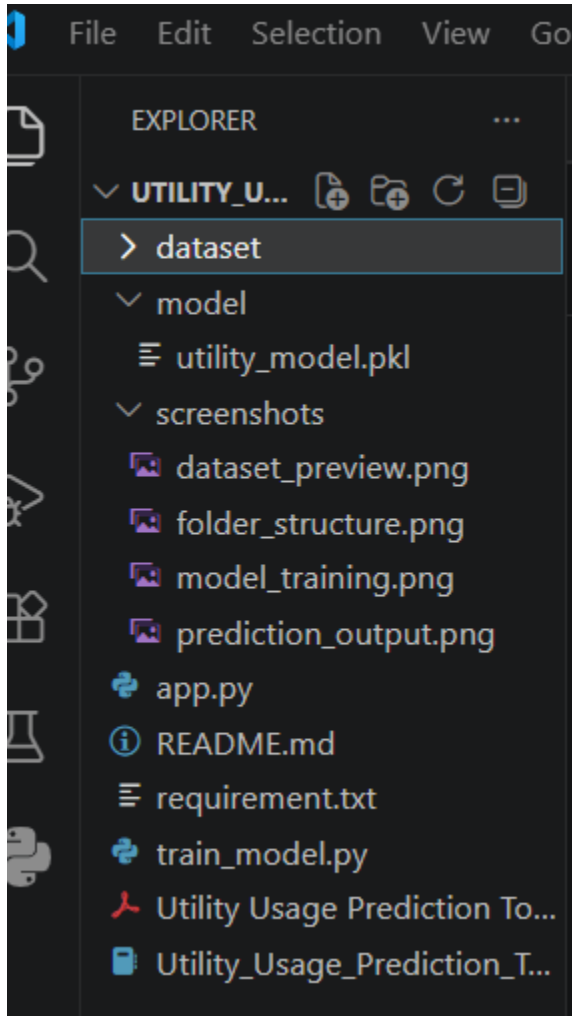
PS C:\Users\Aarti\OneDrive\Desktop\Projects\CodeVedx\Utility_Usage_Prediction_Tool>

4. Project Folder Structure

Description:

The project follows a structured folder organization, making the source code, dataset, and model files easy to understand and maintain.

Screenshot:



ADVANTAGES

- Predicts utility usage quickly and efficiently.
- Reduces manual estimation of utility consumption.
- Supports better planning and resource management.
- Simple and easy-to-use console interface.
- Organized project structure for easy maintenance.
- Demonstrates the practical application of Machine Learning.
- Can be extended for larger datasets and real-world utility management systems.

CONCLUSION

The **Utility Usage Prediction Tool** was successfully developed using **Machine Learning** techniques to predict utility consumption based on historical usage data. The project demonstrates the complete Machine Learning workflow, including data collection, preprocessing, feature selection, model training, evaluation, and prediction.

The system provides an efficient and reliable method for estimating future utility usage, helping users understand consumption patterns and make better decisions regarding resource utilization. The console-based application is simple to use and generates predictions quickly using a trained **Linear Regression** model.

Overall, the project demonstrates the practical application of **Artificial Intelligence and Machine Learning** in utility management and serves as a useful tool for educational and analytical purposes.

FUTURE SCOPE

The Utility Usage Prediction Tool can be enhanced further by implementing the following features:

- Develop a graphical user interface (GUI) or web-based application.
- Integrate real-time utility data from smart meters and IoT devices.
- Use advanced Machine Learning and Deep Learning algorithms to improve prediction accuracy.
- Support multiple utility types such as electricity, water, and gas consumption.
- Integrate cloud storage and database management systems.
- Generate graphical reports and dashboards for better data visualization.
- Develop a mobile application for easy accessibility.
- Implement automatic notifications and usage alerts for users.

REFERENCES

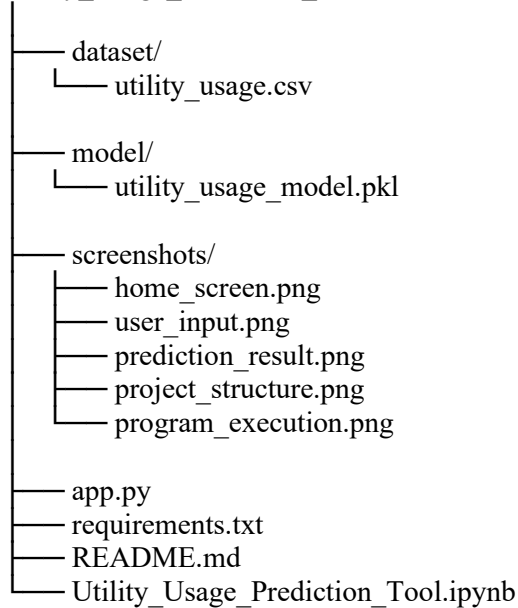
The following resources were referred to during the development of this project:

1. Python Software Foundation. *Python Documentation*. <https://docs.python.org/>
2. Scikit-learn Developers. *Scikit-learn Documentation*. <https://scikit-learn.org/>
3. Pandas Development Team. *Pandas Documentation*. <https://pandas.pydata.org/>
4. NumPy Developers. *NumPy Documentation*. <https://numpy.org/>
5. Joblib Documentation. <https://joblib.readthedocs.io/>
6. Visual Studio Code Documentation. <https://code.visualstudio.com/docs>

APPENDIX

Appendix A – Project Folder Structure

Utility_Usage_Prediction_Tool



Appendix B – Project Modules

The project consists of the following modules:

- Data Collection
- Data Preprocessing
- Feature Selection
- Train-Test Split
- Linear Regression Model Training
- Model Evaluation
- Model Saving using Joblib
- Console-Based Application
- Utility Usage Prediction

Appendix D: Screenshots

The following screenshots are included in the project documentation:

1. User Input Screen
2. Utility Usage Prediction Result
3. Project Folder Structure
4. Program Execution
5. Trained Model File (utility_usage_model.pkl)